

FastAPI 기본 개념 학습 가이드

Python 함수와 타입부터 HTTP/API, FastAPI request/response, CORS, 파일 업로드, 환경변수, 예외 처리
까지 정리한 입문 목차형 자료

읽는 법

1. Python 함수, 타입 힌트, 클래스, 비동기를 먼저 잡습니다.
2. HTTP 요청/응답, method, status code, parameter를 API 언어로 익힙니다.
3. FastAPI endpoint, Pydantic, CORS, 파일 업로드, 환경변수를 작은 예제로 반복합니다.

목차

FastAPI 기본 개념 학습 가이드	5
0. 전체 그림	5
1. Python 핵심 개념: 함수와 타입	6
1.1 함수	6
1.2 함수 매개변수	6
1.3 기본값 매개변수	6
1.4 타입 힌트	7
1.5 기본 타입: str, int, bool, list, dict	7
1.6 Optional	8
1.7 Union	8
1.8 함수와 타입을 FastAPI 관점에서 읽기	9
2. Python 핵심 개념: 객체와 클래스	9
2.1 class	9
2.2 instance	9
2.3 __init__	10
2.4 attribute	10
2.5 method	10
2.6 상속	10
2.7 Pydantic 모델을 객체처럼 읽기	11
3. Python 핵심 개념: 비동기	11
3.1 동기 함수	11
3.2 비동기 함수	12
3.3 async	12
3.4 await	12
3.5 언제 def를 쓰고 언제 async def를 쓸까?	12
3.6 비동기에서 흔한 실수	13
4. HTTP와 API 큰 그림	13
4.1 클라이언트	13
4.2 서버	13
4.3 요청 request	13
4.4 응답 response	13
4.5 URL	14
4.6 endpoint	14
4.7 HTTP method	14
4.8 status code	15
4.9 header	15
4.10 body	15
4.11 JSON	15

4.12 query parameter	16
4.13 path parameter	16
4.14 HTTP 예시 모음	16
5. 상태 코드	16
5.1 200 OK	16
5.2 201 Created	17
5.3 400 Bad Request	17
5.4 401 Unauthorized	17
5.5 403 Forbidden	17
5.6 404 Not Found	18
5.7 422 Validation Error	18
5.8 500 Internal Server Error	18
6. FastAPI 핵심 개념: 기본 구조	19
6.1 설치와 실행	19
6.2 FastAPI app	19
6.3 @app.get	19
6.4 @app.post	19
6.5 endpoint	20
6.6 uvicorn	20
6.7 reload	20
6.8 Swagger UI	20
7. FastAPI 핵심 개념: Request와 Response	21
7.1 request body	21
7.2 response body	21
7.3 JSON response	21
7.4 Pydantic BaseModel	22
7.5 validation	22
7.6 response model	22
8. FastAPI 핵심 개념: Parameter	23
8.1 path parameter	23
8.2 query parameter	23
8.3 body parameter	23
8.4 header	23
8.5 cookie	24
8.6 parameter가 결정되는 규칙	24
9. CORS	25
9.1 CORS란?	25
9.2 왜 React에서 CORS 오류가 날까?	25
9.3 CORSMiddleware	25
9.4 allow_origins	26

9.5 CORS 디버깅 체크리스트	26
10. 파일 업로드	26
10.1 UploadFile	26
10.2 File	27
10.3 multipart/form-data	27
10.4 파일 저장	27
10.5 PDF 읽기	27
11. 환경변수	28
11.1 .env	28
11.2 python-dotenv	28
11.3 os.getenv	29
11.4 OPENAI_API_KEY	29
11.5 DATABASE_URL	29
12. 예외 처리	29
12.1 try / except	29
12.2 HTTPException	30
12.3 status_code	30
12.4 detail	30
12.5 에러 처리 예시: 채팅 API	31
13. React와 연결하는 FastAPI 예제	31
13.1 FastAPI 코드	31
13.2 React fetch 코드	32
13.3 연결이 안 될 때 확인할 것	32
14. 작은 완성 예제: 게시글 API	33
15. RAG 입문 관점에서 보는 FastAPI 흐름	34
15.1 필요한 endpoint	34
15.2 PDF 업로드 흐름	34
15.3 질문 응답 흐름	34
16. 자주 만나는 오류	35
16.1 404가 나옴	35
16.2 422가 나옴	35
16.3 CORS 오류가 나옴	35
16.4 500이 나옴	36
16.5 Swagger UI에서는 되는데 React에서는 안 됨	36
16.6 파일 업로드가 안 됨	36
17. 학습 순서	36
18. 최종 체크리스트	37
참고 자료	37

FastAPI 기본 개념 학습 가이드

작성일: 2026-06-07 대상: Python 문법을 조금 배웠고, React와 연결할 백엔드 API를 만들고 싶은 사람 목표: 아래 키워드를 순서대로 읽으면 FastAPI 기본 개념을 한 번에 정리할 수 있게 만드는 것

이 문서는 FastAPI 문법만 외우는 자료가 아닙니다. FastAPI를 이해하려면 세 가지를 같이 봐야 합니다.

Python 문법	함수를 어떻게 만들고 타입을 어떻게 표현하는가
HTTP / API	클라이언트와 서버가 어떤 약속으로 데이터를 주고받는가
FastAPI	Python 함수와 HTTP API를 어떻게 연결하는가

React와 FastAPI를 연결할 때는 React가 클라이언트, FastAPI가 서버 역할을 합니다. React는 `fetch`나 `axios`로 요청을 보내고, FastAPI는 JSON 응답을 돌려줍니다.

0. 전체 그림

FastAPI 앱 하나는 보통 이런 흐름으로 동작합니다.

```
브라우저 / React
|
| HTTP 요청
v
FastAPI 서버
|
| Python 함수 실행
v
데이터 처리 / DB / AI 모델 / 파일 읽기
|
| HTTP 응답
v
브라우저 / React
```

예를 들어 React에서 채팅 메시지를 보낸다고 생각해 봅시다.

```
POST /chat
body: {"message": "안녕"}
```

FastAPI는 이 요청을 받아 Python 함수에 넘깁니다.

```
@app.post("/chat")
def chat(request: ChatRequest):
    return {"answer": request.message}
```

그리고 React는 이런 응답을 받습니다.

```
{"answer": "안녕"}
```

이 흐름을 이해하면 `@app.post`, `BaseModel`, `request body`, `JSON`, `CORS`, `status code` 같은 키워드가 따로따로 보이지 않고 하나의 연결된 구조로 보입니다.

1. Python 핵심 개념: 함수와 타입

FastAPI 코드는 대부분 "HTTP 요청을 받는 Python 함수"로 작성됩니다. 그래서 함수, 매개변수, 타입 힌트를 먼저 알아야 합니다.

1.1 함수

함수는 반복해서 사용할 코드를 이름으로 묶은 것입니다.

```
def say_hello():  
    return "안녕하세요"  
  
result = say_hello()  
print(result)
```

`def`는 함수를 정의한다는 뜻입니다. 함수 이름 뒤의 괄호 안에는 필요한 값을 받을 수 있고, `return`은 함수 실행 결과를 바깥으로 돌려줍니다.

FastAPI에서 endpoint도 결국 함수입니다.

```
@app.get("/")  
def root():  
    return {"message": "Hello FastAPI"}
```

브라우저가 / 주소로 요청하면 FastAPI가 `root()` 함수를 실행하고, 함수가 반환한 딕셔너리를 JSON 응답으로 바꿔 줍니다.

1.2 함수 매개변수

매개변수는 함수 안으로 들어오는 값입니다.

```
def greet(name):  
    return f"안녕하세요, {name}님"  
  
print(greet("지민"))
```

`name`은 매개변수이고, "지민"은 함수를 호출할 때 넣은 실제 값입니다.

FastAPI에서는 매개변수가 HTTP 요청의 일부와 연결됩니다.

```
@app.get("/posts/{post_id}")  
def read_post(post_id: int):  
    return {"post_id": post_id}
```

`/posts/3`으로 요청하면 `post_id`에 3이 들어갑니다.

1.3 기본값 매개변수

매개변수에 기본값을 줄 수 있습니다.

```
def search(keyword: str = "fastapi"):  
    return f"{keyword} 검색"
```

호출할 때 값을 넣지 않으면 기본값이 사용됩니다.

```
search()      # "fastapi 검색"
search("rag") # "rag 검색"
```

FastAPI에서는 기본값이 있는 매개변수가 query parameter로 자주 쓰입니다.

```
@app.get("/posts")
def list_posts(keyword: str = "", page: int = 1):
    return {"keyword": keyword, "page": page}
```

요청 URL은 이렇게 됩니다.

```
GET /posts?keyword=rag&page=2
```

1.4 타입 힌트

타입 힌트는 변수나 매개변수에 어떤 종류의 값이 들어와야 하는지 표시하는 문법입니다.

```
def chat(message: str) -> dict:
    return {"answer": message}
```

message: str은 message가 문자열이라는 뜻입니다. -> dict는 이 함수가 딕셔너리를 반환한다는 뜻입니다.

Python은 타입 힌트를 적었다고 해서 Java나 TypeScript처럼 완전히 엄격하게 컴파일하지는 않습니다. 하지만 FastAPI에서는 타입 힌트가 매우 중요합니다. FastAPI는 타입 힌트를 보고 요청 값을 변환하고, 검증하고, Swagger UI 문서를 만듭니다.

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

/items/3은 정상입니다. 하지만 /items/abc는 abc를 정수로 바꿀 수 없기 때문에 FastAPI가 자동으로 검증 오류를 반환합니다.

1.5 기본 타입: str, int, bool, list, dict

FastAPI에서 자주 보는 Python 기본 타입은 아래와 같습니다.

```
str 문자열, 예: "hello"
int 정수, 예: 3
bool 참/거짓, 예: True, False
list 배열처럼 여러 값을 담는 자료형
dict key-value 형태의 객체 같은 자료형
```

예시입니다.

```
name: str = "FastAPI"
count: int = 3
is_public: bool = True
tags: list[str] = ["python", "api"]
post: dict = {"title": "첫 글", "views": 10}
```

FastAPI endpoint에서는 이렇게 섞어서 사용합니다.

```
@app.get("/search")
def search(keyword: str, limit: int = 10, exact: bool = False):
    return {
        "keyword": keyword,
        "limit": limit,
        "exact": exact,
    }
```

요청 예시는 아래와 같습니다.

```
GET /search?keyword=fastapi&limit=5&exact=true
```

FastAPI는 "5"라는 URL 문자열을 int인 5로 바꾸고, "true"를 bool인 True로 바꿔 줍니다.

1.6 Optional

Optional은 값이 있을 수도 있고 없을 수도 있다는 뜻입니다.

```
from typing import Optional

def find_user(name: Optional[str] = None):
    if name is None:
        return "이름이 없습니다"
    return f"{name} 검색"
```

요즘 Python에서는 아래처럼 쓰기도 합니다.

```
def find_user(name: str | None = None):
    ...
```

FastAPI에서는 선택적 query parameter를 표현할 때 자주 씁니다.

```
from typing import Optional
from fastapi import FastAPI

app = FastAPI()

@app.get("/posts")
def list_posts(keyword: Optional[str] = None):
    return {"keyword": keyword}
```

아래 두 요청 모두 가능합니다.

```
GET /posts
GET /posts?keyword=fastapi
```

1.7 Union

Union은 여러 타입 중 하나가 올 수 있다는 뜻입니다.

```
from typing import Union

def stringify(value: Union[str, int]):
    return str(value)
```


Python 3.10 이상에서는 아래처럼 쓸 수 있습니다.

```
def stringify(value: str | int):
    return str(value)
```

FastAPI 입문 단계에서는 Union을 자주 직접 쓸 일은 많지 않습니다. 하지만 Optional[str]이 사실상 Union[str, None]과 같은 의미라는 점은 알아두면 좋습니다.

```
from typing import Optional, Union

name1: Optional[str] = None
name2: Union[str, None] = None
```

1.8 함수와 타입을 FastAPI 관점에서 읽기

아래 코드를 한 줄씩 읽어 봅시다.

```
def chat(message: str) -> dict:
    return {"answer": message}
```

```
def chat          chat이라는 함수를 만든다
message: str      message는 문자열이다
-> dict           반환값은 딕셔너리다
return {"answer": ...} 딕셔너리를 반환한다
```

FastAPI에서 이 함수가 endpoint가 되면 "문자열을 받아 JSON 객체를 돌려주는 API"처럼 생각할 수 있습니다.

2. Python 핵심 개념: 객체와 클래스

처음 FastAPI를 배울 때 클래스를 깊게 파고들 필요는 없습니다. 하지만 Pydantic의 BaseModel, SQLAlchemy의 모델, 설정 클래스, 의존성 주입을 만나면 클래스 개념이 계속 나옵니다.

2.1 class

class는 객체를 만들기 위한 설계도입니다.

```
class User:
    pass
```

이 코드는 User라는 설계도를 만든 것입니다. 아직 실제 사용자는 만들어지지 않았습니다.

2.2 instance

클래스로 실제 만들어진 객체를 instance라고 합니다.

```
class User:
    pass

user1 = User()
user2 = User()
```

user1, user2는 모두 User 클래스로 만든 instance입니다.

2.3 __init__

`__init__`은 instance가 만들어질 때 자동으로 실행되는 초기화 함수입니다.

```
class User:
    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

user = User("지민", 20)
```

`User("지민", 20)`을 실행하면 `__init__`이 호출되고, `name`, `age` 값이 객체 안에 저장됩니다.

2.4 attribute

attribute는 객체가 가지고 있는 값입니다.

```
print(user.name)
print(user.age)
```

여기서 `name`, `age`가 attribute입니다. JavaScript 객체의 `user.name`과 비슷하게 읽으면 됩니다.

2.5 method

method는 객체 안에 들어 있는 함수입니다.

```
class User:
    def __init__(self, name: str):
        self.name = name

    def greet(self):
        return f"안녕하세요, {self.name}님"

user = User("지민")
print(user.greet())
```

`greet`는 `User` 객체가 사용할 수 있는 method입니다.

2.6 상속

상속은 기존 클래스의 기능을 물려받아 새 클래스를 만드는 것입니다.

```
class Animal:
    def speak(self):
        return "소리를 냅니다"

class Dog(Animal):
    pass

dog = Dog()
print(dog.speak())
```

FastAPI에서 가장 많이 보는 상속 예시는 Pydantic 모델입니다.

```
from pydantic import BaseModel
```

```
class ChatRequest(BaseModel):
    message: str
```

ChatRequest는 BaseModel을 상속합니다. 그래서 message의 타입 검증, JSON 변환, 문서 생성 같은 기능을 사용할 수 있습니다.

2.7 Pydantic 모델을 객체처럼 읽기

FastAPI에서 request body를 받을 때는 보통 Pydantic 모델을 만듭니다.

```
from pydantic import BaseModel

class PostCreate(BaseModel):
    title: str
    content: str
    published: bool = True
```

이 모델은 아래 JSON을 받을 수 있습니다.

```
{
  "title": "FastAPI 시작",
  "content": "첫 번째 글입니다",
  "published": true
}
```

endpoint 안에서는 객체처럼 접근합니다.

```
@app.post("/posts")
def create_post(post: PostCreate):
    return {
        "title": post.title,
        "content": post.content,
        "published": post.published,
    }
```

여기서 post.title은 JavaScript의 post.title처럼 읽으면 됩니다.

3. Python 핵심 개념: 비동기

FastAPI 예제에는 async def와 await가 자주 나옵니다. 처음에는 "기다리는 일이 많은 코드에서 서버가 멈추지 않게 도와주는 문법"으로 이해하면 됩니다.

3.1 동기 함수

동기 함수는 한 줄이 끝나야 다음 줄로 넘어갑니다.

```
def download_file():
    data = slow_network_call()
    return data
```

slow_network_call()이 오래 걸리면 그 동안 다음 줄로 넘어가지 못합니다.

3.2 비동기 함수

비동기 함수는 `async def`로 만듭니다.

```
async def download_file():
    data = await slow_network_call()
    return data
```

`await`는 "이 작업이 끝날 때까지 기다리되, 기다리는 동안 다른 일을 할 기회를 준다"는 의미입니다.

3.3 async

`async`는 이 함수가 비동기 함수라는 표시입니다.

```
async def read_root():
    return {"message": "Hello"}
```

FastAPI에서는 endpoint를 `def`로 만들어도 되고 `async def`로 만들어도 됩니다.

```
@app.get("/sync")
def sync_endpoint():
    return {"type": "sync"}

@app.get("/async")
async def async_endpoint():
    return {"type": "async"}
```

3.4 await

`await`는 비동기 작업의 결과가 필요할 때 사용합니다. `await`는 `async def` 함수 안에서만 사용할 수 있습니다.

```
async def get_answer():
    result = await call_ai_api()
    return result
```

파일, 데이터베이스, 외부 API 호출처럼 기다리는 시간이 있는 작업에서 많이 보입니다.

3.5 언제 def를 쓰고 언제 async def를 쓸까?

입문 단계에서는 이렇게 생각하면 충분합니다.

- 일반 계산, 단순한 딕셔너리 반환: `def`로 시작해도 됩니다.
- 비동기 라이브러리 호출이 필요함: `async def`와 `await`를 씁니다.
- `await some_function()`을 써야 한다면 그 endpoint는 `async def`여야 합니다.

예시입니다.

```
@app.get("/health")
def health_check():
    return {"ok": True}
```

```
@app.post("/chat")
async def chat(request: ChatRequest):
    answer = await call_llm(request.message)
```

```
return {"answer": answer}
```

3.6 비동기에서 흔한 실수

`await`를 빼먹으면 실제 결과가 아니라 `coroutine` 객체가 반환될 수 있습니다.

```
async def chat(request: ChatRequest):
    answer = call_llm(request.message) # await가 빠짐
    return {"answer": answer}
```

비동기 함수 호출 결과가 필요하면 아래처럼 써야 합니다.

```
async def chat(request: ChatRequest):
    answer = await call_llm(request.message)
    return {"answer": answer}
```

4. HTTP와 API 큰 그림

FastAPI는 HTTP API를 쉽게 만드는 도구입니다. 그래서 FastAPI 문법을 배우기 전에 HTTP 단어를 알아야 합니다.

4.1 클라이언트

클라이언트는 요청을 보내는 쪽입니다. 웹 개발에서는 보통 브라우저나 React 앱이 클라이언트입니다.

```
React 앱 -> FastAPI 서버로 요청
```

사용자가 버튼을 누르면 React는 `fetch`나 `axios`로 서버에 요청을 보냅니다.

4.2 서버

서버는 요청을 받고 응답을 돌려주는 쪽입니다. FastAPI 앱은 서버 프로그램입니다.

```
FastAPI 서버 -> 요청을 처리하고 JSON 응답 반환
```

서버는 DB에서 데이터를 가져오거나, 파일을 저장하거나, AI API를 호출한 뒤 결과를 응답으로 보냅니다.

4.3 요청 request

`request`는 클라이언트가 서버에 보내는 메시지입니다.

```
GET /posts
POST /chat
DELETE /posts/3
```

`request`에는 보통 URL, HTTP method, header, body가 포함될 수 있습니다.

4.4 응답 response

`response`는 서버가 클라이언트에 돌려주는 메시지입니다.

```
{
```

```
{
  "id": 3,
  "title": "FastAPI 입문"
}
```

response에는 status code, header, body가 포함될 수 있습니다.

4.5 URL

URL은 서버의 특정 자원을 가리키는 주소입니다.

```
http://localhost:8000/posts/3?keyword=fastapi
```

나눠 보면 아래와 같습니다.

```
http://localhost:8000  서버 주소
/posts/3              path
?keyword=fastapi      query string
```

4.6 endpoint

endpoint는 API에서 요청을 받을 수 있는 특정 주소와 동작의 조합입니다.

```
GET /posts
POST /chat
GET /posts/3
DELETE /posts/3
```

FastAPI에서는 endpoint를 decorator와 함수로 만듭니다.

```
@app.get("/posts")
def list_posts():
    return [{"id": 1, "title": "첫 글"}]
```

4.7 HTTP method

HTTP method는 요청의 목적을 나타냅니다.

```
GET   데이터를 조회한다
POST  새 데이터를 만든다, 명령을 보낸다
PUT   전체 데이터를 교체한다
PATCH 일부 데이터를 수정한다
DELETE 데이터를 삭제한다
```

게시글 API를 예로 들면 아래처럼 나눌 수 있습니다.

GET /posts	게시글 목록 조회
POST /posts	게시글 생성
GET /posts/3	3번 게시글 조회
PUT /posts/3	3번 게시글 전체 수정
PATCH /posts/3	3번 게시글 일부 수정
DELETE /posts/3	3번 게시글 삭제

채팅 API는 보통 사용자의 메시지를 body에 담아 보내므로 **POST**를 많이 씁니다.

```
POST /chat
```

4.8 status code

status code는 요청 결과를 숫자로 표현한 것입니다.

```
2xx 성공
4xx 클라이언트 요청 문제
5xx 서버 내부 문제
```

FastAPI는 정상 응답이면 보통 200 OK를 돌려줍니다. 데이터 생성 API라면 201 Created를 명시할 수 있습니다.

```
from fastapi import status

@app.post("/posts", status_code=status.HTTP_201_CREATED)
def create_post():
    return {"message": "created"}
```

4.9 header

header는 요청이나 응답에 붙는 부가 정보입니다.

```
Content-Type: application/json
Authorization: Bearer token...
```

React에서 JSON을 보낼 때는 보통 아래 header를 넣습니다.

```
headers: {
  "Content-Type": "application/json"
}
```

로그인 이후 API에서는 인증 토큰을 header에 담아 보내는 경우가 많습니다.

4.10 body

body는 요청이나 응답의 실제 데이터 부분입니다.

```
{
  "message": "안녕하세요"
}
```

GET 요청은 보통 body를 사용하지 않고, POST, PUT, PATCH 요청에서 body를 자주 사용합니다.

4.11 JSON

JSON은 클라이언트와 서버가 데이터를 주고받을 때 가장 많이 쓰는 형식입니다.

```
{
  "title": "FastAPI",
  "tags": ["python", "backend"],
  "published": true
}
```

Python의 딕셔너리와 비슷하게 생겼지만, JSON은 문자열 기반 데이터 형식입니다. FastAPI는 Python의 dict, list, Pydantic 모델을 JSON 응답으로 바꿔 줍니다.

4.12 query parameter

query parameter는 URL의 ? 뒤에 붙는 값입니다. 검색, 필터, 페이지 번호처럼 선택적인 조건에 많이 씁니다.

```
GET /posts?keyword=rag&page=2
```

FastAPI에서는 함수 매개변수에 기본값을 주면 query parameter로 해석되는 경우가 많습니다.

```
@app.get("/posts")
def list_posts(keyword: str = "", page: int = 1):
    return {"keyword": keyword, "page": page}
```

4.13 path parameter

path parameter는 URL 경로 안에 들어가는 값입니다. 특정 자원을 가리킬 때 많이 씁니다.

```
GET /posts/3
DELETE /posts/3
```

FastAPI에서는 {post_id}처럼 표시합니다.

```
@app.get("/posts/{post_id}")
def read_post(post_id: int):
    return {"post_id": post_id}
```

4.14 HTTP 예시 모음

아래 예시는 입문 단계에서 꼭 익숙해져야 합니다.

```
GET /posts
의미: 게시글 목록을 가져온다

POST /chat
의미: 채팅 메시지를 서버에 보낸다

GET /posts/3
의미: 3번 게시글 하나를 가져온다

DELETE /posts/3
의미: 3번 게시글을 삭제한다
```

5. 상태 코드

상태 코드는 API를 디버깅할 때 가장 먼저 봐야 하는 신호입니다.

5.1 200 OK

200 OK는 요청이 성공했다는 뜻입니다.


```
GET /posts/3 -> 200 OK
```

FastAPI에서 일반적으로 값을 반환하면 200 응답이 나옵니다.

```
@app.get("/health")
def health_check():
    return {"ok": True}
```

5.2 201 Created

201 Created는 새 자원이 만들어졌다는 뜻입니다.

```
from fastapi import status

@app.post("/posts", status_code=status.HTTP_201_CREATED)
def create_post():
    return {"message": "created"}
```

게시글 생성, 회원가입, 파일 업로드처럼 새 데이터가 생기는 API에서 잘 어울립니다.

5.3 400 Bad Request

400 Bad Request는 요청 자체가 잘못되었다는 뜻입니다.

잘못된 JSON 형식
허용하지 않는 요청 조합
비즈니스 규칙에 맞지 않는 요청

FastAPI에서는 직접 `HTTPException`으로 400을 반환할 수 있습니다.

```
from fastapi import HTTPException

if len(message) == 0:
    raise HTTPException(status_code=400, detail="message는 비어 있을 수 없습니다")
```

5.4 401 Unauthorized

401 Unauthorized는 인증이 필요하거나 인증 정보가 올바르지 않다는 뜻입니다.

토큰이 없음
토큰이 만료됨
토큰 형식이 잘못됨

로그인 기반 API에서 자주 봅니다.

5.5 403 Forbidden

403 Forbidden은 인증은 되었지만 권한이 없다는 뜻입니다.

로그인은 했지만 관리자 권한이 없음
다른 사용자의 글을 삭제하려고 함

401은 "누구인지 확인이 안 됨"에 가깝고, 403은 "누구인지는 알지만 허용되지 않음"에 가깝습니다.

5.6 404 Not Found

404 Not Found는 요청한 자원을 찾을 수 없다는 뜻입니다.

```
GET /posts/999 -> 404 Not Found
```

```
post = find_post(post_id)
if post is None:
    raise HTTPException(status_code=404, detail="게시글을 찾을 수 없습니다")
```

5.7 422 Validation Error

422 Validation Error는 FastAPI에서 정말 자주 만나는 상태 코드입니다. 요청 형식은 서버에 도착했지만, 타입이나 필수 값 검증에 실패했다는 뜻입니다.

예를 들어 아래 endpoint가 있다고 합시다.

```
@app.get("/posts/{post_id}")
def read_post(post_id: int):
    return {"post_id": post_id}
```

이 요청은 정상입니다.

```
GET /posts/3
```

이 요청은 422가 날 수 있습니다.

```
GET /posts/abc
```

abc는 int로 변환할 수 없기 때문입니다.

Pydantic 모델에서 필수 필드를 빼먹어도 422가 납니다.

```
class ChatRequest(BaseModel):
    message: str
```

```
{}
```

message가 없기 때문에 검증 실패입니다.

5.8 500 Internal Server Error

500 Internal Server Error는 서버 코드 내부에서 예상하지 못한 오류가 났다는 뜻입니다.

```
DB 연결 실패
파일 경로 오류
None 값을 잘못 사용
외부 API 호출 실패를 처리하지 않음
```

500은 사용자 요청 문제가 아니라 서버 코드 문제가 많습니다. 로그를 확인하고 try / except나 예외 처리를 보강해야 합니다.

6. FastAPI 핵심 개념: 기본 구조

이제 FastAPI 코드를 봅시다.

6.1 설치와 실행

일반적인 설치 방법은 아래처럼 합니다.

```
pip install fastapi uvicorn
```

개발 서버 실행은 보통 Uvicorn으로 합니다.

```
uvicorn main:app --reload
```

의미는 아래와 같습니다.

```
main    main.py 파일
app     main.py 안에 있는 FastAPI 객체 이름
--reload 코드가 바뀌면 개발 서버를 자동 재시작
```

최근 FastAPI 문서에서는 FastAPI CLI의 `fastapi dev` 명령도 볼 수 있습니다. 하지만 입문 단계에서는 `uvicorn main:app --reload`를 먼저 익히면 충분합니다.

6.2 FastAPI app

가장 작은 FastAPI 앱은 아래처럼 생겼습니다.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello FastAPI"}
```

`app = FastAPI()`는 FastAPI 애플리케이션 객체를 만드는 코드입니다. 이 `app`에 endpoint를 등록합니다.

6.3 @app.get

`@app.get("/")`은 GET / 요청이 들어오면 바로 아래 함수를 실행하라는 뜻입니다.

```
@app.get("/")
def root():
    return {"message": "Hello"}
```

이런 `@...` 문법을 decorator라고 합니다. FastAPI에서는 decorator가 HTTP method와 URL path를 연결합니다.

6.4 @app.post

`@app.post("/chat")`은 POST /chat 요청을 처리합니다.

```
@app.post("/chat")
def chat():
    return {"answer": "안녕하세요"}
```

POST는 body에 데이터를 담아 보낼 때 자주 씁니다. 채팅, 회원가입, 글 작성, 파일 업로드 같은 API에서 많이 사용됩니다.

6.5 endpoint

FastAPI에서 endpoint는 decorator와 함수가 합쳐진 단위입니다.

```
@app.get("/posts")
def list_posts():
    return [{"id": 1, "title": "첫 글"}]
```

여기서 endpoint는 GET /posts입니다. 함수 이름 list_posts는 Python 코드 내부 이름이고, 외부 클라이언트가 호출하는 주소는 /posts입니다.

6.6 uvicorn

Uvicorn은 FastAPI 앱을 실제 HTTP 서버로 실행해 주는 ASGI 서버입니다.

```
uvicorn main:app --reload
```

서버가 실행되면 보통 아래 주소에서 확인합니다.

```
http://127.0.0.1:8000
http://localhost:8000
```

6.7 reload

--reload는 개발할 때 편리한 옵션입니다.

```
uvicorn main:app --reload
```

파일을 수정하면 서버가 자동으로 재시작됩니다. 운영 서버에서는 보통 --reload를 쓰지 않고, 개발 중에만 사용합니다.

6.8 Swagger UI

FastAPI는 자동 API 문서를 만들어 줍니다.

```
http://localhost:8000/docs
```

이 주소로 들어가면 Swagger UI를 볼 수 있습니다. Swagger UI에서는 endpoint 목록, parameter, request body, response schema를 확인하고 직접 API를 테스트할 수 있습니다.

또 다른 문서 화면은 아래 주소입니다.

```
http://localhost:8000/redoc
```

입문 단계에서는 /docs를 자주 열어두고 테스트하면 좋습니다.

7. FastAPI 핵심 개념: Request와 Response

FastAPI의 핵심은 request를 Python 값으로 받고, Python 값을 response로 돌려주는 것입니다.

7.1 request body

request body는 클라이언트가 서버에 보내는 본문 데이터입니다.

```
{
  "message": "안녕하세요"
}
```

FastAPI에서는 Pydantic 모델로 body 구조를 정의합니다.

```
from pydantic import BaseModel

class ChatRequest(BaseModel):
    message: str
```

그리고 endpoint 매개변수로 받습니다.

```
@app.post("/chat")
def chat(request: ChatRequest):
    return {"answer": request.message}
```

7.2 response body

response body는 서버가 클라이언트에 돌려주는 본문 데이터입니다.

```
@app.post("/chat")
def chat(request: ChatRequest):
    return {
        "answer": f"입력한 메시지: {request.message}"
    }
```

FastAPI는 Python 딕셔너리를 JSON response body로 바꿔 줍니다.

7.3 JSON response

FastAPI endpoint에서 dict나 list를 반환하면 기본적으로 JSON 응답이 됩니다.

```
@app.get("/posts")
def list_posts():
    return [
        {"id": 1, "title": "첫 글"},
        {"id": 2, "title": "두 번째 글"},
    ]
```

응답은 이런 JSON입니다.

```
[
  {"id": 1, "title": "첫 글"},
  {"id": 2, "title": "두 번째 글"}
]
```

7.4 Pydantic BaseModel

BaseModel은 데이터의 모양을 정의하고 검증하는 Pydantic 클래스입니다.

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    email: str
    password: str
    age: int | None = None
```

이 모델은 다음 역할을 합니다.

- 어떤 필드가 필요한지 정의합니다.
- 각 필드의 타입을 검증합니다.
- Swagger UI에 request body 예시와 schema를 보여줍니다.
- endpoint 안에서 객체처럼 접근할 수 있게 해 줍니다.

7.5 validation

validation은 들어온 데이터가 약속한 형식에 맞는지 검사하는 것입니다.

```
class ChatRequest(BaseModel):
    message: str
```

아래 요청은 정상입니다.

```
{"message": "안녕"}
```

아래 요청은 실패합니다.

```
{"message": 123}
```

FastAPI와 Pydantic은 타입을 기준으로 검증합니다. 검증에 실패하면 보통 422 Validation Error가 반환됩니다.

7.6 response model

응답 형식도 Pydantic 모델로 정의할 수 있습니다.

```
class ChatResponse(BaseModel):
    answer: str

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest):
    return {"answer": request.message}
```

response_model을 사용하면 응답 데이터의 모양을 문서화하고, 의도하지 않은 필드가 응답에 섞이는 일을 줄일 수 있습니다.

8. FastAPI 핵심 개념: Parameter

FastAPI는 함수 매개변수의 위치와 타입을 보고 request에서 값을 가져옵니다.

8.1 path parameter

path parameter는 URL 경로 안에 있는 값입니다.

```
@app.get("/posts/{post_id}")
def read_post(post_id: int):
    return {"post_id": post_id}
```

요청 예시입니다.

```
GET /posts/3
```

post_id에는 3이 들어갑니다.

8.2 query parameter

query parameter는 ? 뒤에 붙는 값입니다.

```
@app.get("/posts")
def list_posts(keyword: str = "", limit: int = 10):
    return {"keyword": keyword, "limit": limit}
```

요청 예시입니다.

```
GET /posts?keyword=rag&limit=5
```

keyword에는 "rag", limit에는 5가 들어갑니다.

8.3 body parameter

body parameter는 request body에서 오는 값입니다. Pydantic 모델을 사용하면 body로 해석됩니다.

```
class PostCreate(BaseModel):
    title: str
    content: str

@app.post("/posts")
def create_post(post: PostCreate):
    return {"title": post.title, "content": post.content}
```

요청 body는 JSON입니다.

```
{
  "title": "FastAPI",
  "content": "본문입니다"
}
```

8.4 header

header 값도 받을 수 있습니다.

```
from fastapi import Header

@app.get("/me")
def read_me(authorization: str | None = Header(default=None)):
    return {"authorization": authorization}
```

요청 header 예시입니다.

```
Authorization: Bearer abc.def.ghi
```

인증 토큰은 보통 header에 담겨 옵니다.

8.5 cookie

cookie 값도 받을 수 있습니다.

```
from fastapi import Cookie

@app.get("/visit")
def read_visit(session_id: str | None = Cookie(default=None)):
    return {"session_id": session_id}
```

입문 단계에서 cookie는 자주 직접 쓰지 않을 수 있지만, 로그인 세션 방식에서는 중요해집니다.

8.6 parameter가 결정되는 규칙

입문 단계에서는 아래 규칙으로 읽으면 됩니다.

```
URL path에 {name}이 있고 함수 매개변수도 name이면 path parameter
기본 타입 매개변수이고 path에 없으면 query parameter
Pydantic BaseModel 타입이면 request body
Header(...)를 쓰면 header
Cookie(...)를 쓰면 cookie
```

예시입니다.

```
from fastapi import Header
from pydantic import BaseModel

class PostUpdate(BaseModel):
    title: str
    content: str

@app.patch("/posts/{post_id}")
def update_post(
    post_id: int,
    post: PostUpdate,
    authorization: str | None = Header(default=None),
):
    return {
        "post_id": post_id,
        "title": post.title,
        "authorization": authorization,
    }
```

이 endpoint는 세 곳에서 값을 받습니다.


```
post_id    path parameter
post       request body
authorization header
```

9. CORS

React와 FastAPI를 연결할 때 거의 반드시 만나는 개념이 CORS입니다.

9.1 CORS란?

CORS는 Cross-Origin Resource Sharing의 약자입니다. 브라우저가 보안 때문에 "다른 출처(origin)"로 요청을 보낼 때 검사하는 규칙입니다.

origin은 보통 아래 세 가지가 합쳐진 것입니다.

```
프로토콜 + 도메인 + 포트
```

예를 들어 아래 두 주소는 origin이 다릅니다.

```
http://localhost:5173 React 개발 서버
http://localhost:8000 FastAPI 서버
```

도메인이 둘 다 `localhost`여도 포트가 다르면 다른 origin입니다.

9.2 왜 React에서 CORS 오류가 날까?

React 개발 서버가 `localhost:5173`에서 실행되고, FastAPI가 `localhost:8000`에서 실행된다고 합시다.

```
React -> http://localhost:8000/chat
5173    8000
```

브라우저는 다른 origin으로 요청을 보내는 상황이라고 판단합니다. 이때 FastAPI 서버가 "localhost:5173에서 오는 요청을 허용한다"고 알려주지 않으면 브라우저가 응답을 막습니다.

중요한 점은 CORS가 FastAPI 함수가 실행되지 않는 문제와 항상 같은 것은 아니라는 점입니다. 서버가 응답했더라도 브라우저가 보안 정책 때문에 React 코드에 응답을 넘겨주지 않을 수 있습니다.

9.3 CORSMiddleware

FastAPI에서는 `CORSMiddleware`를 추가해서 CORS를 설정합니다.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost:5173",
    "http://127.0.0.1:5173",
]

app.add_middleware(
    CORSMiddleware,
```

```
allow_origins=origins,
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)
```

9.4 allow_origins

`allow_origins`는 어떤 origin의 요청을 허용할지 정하는 옵션입니다.

```
allow_origins=[
    "http://localhost:5173",
]
```

React Vite 개발 서버는 보통 `localhost:5173`을 사용합니다. FastAPI는 보통 `localhost:8000`입니다.

```
React http://localhost:5173
API http://localhost:8000
```

개발 중에는 특정 origin을 명확히 적는 습관이 좋습니다.

9.5 CORS 디버깅 체크리스트

CORS 오류가 나면 아래를 확인합니다.

- React 앱 주소가 `allow_origins`에 정확히 들어 있는지 확인합니다.
- `localhost`와 `127.0.0.1`은 서로 다른 origin으로 취급될 수 있으니 둘 다 필요한지 확인합니다.
- 포트 번호가 맞는지 확인합니다.
- FastAPI 서버를 재시작했는지 확인합니다.
- 요청 URL이 진짜 FastAPI 서버 주소인지 확인합니다.

10. 파일 업로드

RAG를 만들려면 사용자가 PDF를 업로드하고, 서버가 파일을 저장하거나 읽어야 합니다. FastAPI의 파일 업로드는 이때 필요합니다.

10.1 UploadFile

`UploadFile`은 업로드된 파일을 표현하는 FastAPI 타입입니다.

```
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    return {
        "filename": file.filename,
        "content_type": file.content_type,
    }
```

file.filename에는 원래 파일 이름이 들어 있고, file.content_type에는 파일 형식 정보가 들어 있습니다.

10.2 File

File(...)은 이 매개변수가 업로드 파일이라는 것을 FastAPI에 알려줍니다.

```
file: UploadFile = File(...)
```

...은 이 값이 필수라는 의미로 자주 쓰입니다. 파일을 보내지 않으면 검증 오류가 납니다.

10.3 multipart/form-data

파일 업로드 요청은 일반 JSON body가 아니라 multipart/form-data 형식을 사용합니다.

React에서 파일을 보낼 때는 보통 FormData를 씁니다.

```
const formData = new FormData();
formData.append("file", selectedFile);

await fetch("http://localhost:8000/upload", {
  method: "POST",
  body: formData,
});
```

주의할 점은 FormData를 보낼 때는 보통 Content-Type을 직접 application/json으로 설정하지 않는다는 것입니다. 브라우저가 multipart boundary를 포함한 적절한 header를 자동으로 만듭니다.

10.4 파일 저장

업로드된 파일을 서버에 저장할 수 있습니다.

```
from pathlib import Path
from fastapi import FastAPI, UploadFile, File

app = FastAPI()
UPLOAD_DIR = Path("uploads")
UPLOAD_DIR.mkdir(exist_ok=True)

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    save_path = UPLOAD_DIR / file.filename

    content = await file.read()
    save_path.write_bytes(content)

    return {"filename": file.filename, "saved_to": str(save_path)}
```

실제 서비스에서는 파일 이름 충돌, 확장자 검사, 파일 크기 제한, 악성 파일 검사 등을 추가해야 합니다.

10.5 PDF 읽기

RAG에서는 업로드된 PDF에서 텍스트를 추출한 뒤, 문서를 잘라 embedding하고 검색에 사용합니다.

입문용 흐름은 아래와 같습니다.

1. PDF 파일 업로드
2. 서버에 저장

3. PDF 텍스트 추출
4. 문단 단위로 나누기
5. embedding 만들기
6. vector DB에 저장
7. 질문이 들어오면 관련 문단 검색
8. LLM에 문맥과 질문을 같이 전달

PDF 읽기 라이브러리는 프로젝트에 따라 다릅니다. 예를 들어 `pypdf`, `pdfplumber`, `PyMuPDF` 등을 사용할 수 있습니다.

간단한 예시는 아래와 같습니다.

```
from pypdf import PdfReader

def read_pdf_text(path: str) -> str:
    reader = PdfReader(path)
    texts = []

    for page in reader.pages:
        texts.append(page.extract_text() or "")

    return "\n".join(texts)
```

그리고 업로드 endpoint 안에서 저장 후 읽을 수 있습니다.

```
@app.post("/upload-pdf")
async def upload_pdf(file: UploadFile = File(...)):
    save_path = UPLOAD_DIR / file.filename
    content = await file.read()
    save_path.write_bytes(content)

    text = read_pdf_text(str(save_path))
    return {"filename": file.filename, "text_length": len(text)}
```

11. 환경변수

API Key, DB URL 같은 민감한 값은 코드에 직접 적지 않는 것이 좋습니다. 이런 값은 환경변수로 관리합니다.

11.1 .env

`.env` 파일은 개발 환경에서 환경변수를 관리하기 위한 파일입니다.

```
OPENAI_API_KEY=sk-...
DATABASE_URL=postgresql://user:password@localhost:5432/app
```

`.env` 파일에는 비밀값이 들어가므로 Git에 올리지 않는 것이 원칙입니다. 보통 `.gitignore`에 `.env`를 추가합니다.

11.2 python-dotenv

`python-dotenv`는 `.env` 파일을 읽어서 환경변수로 올려주는 라이브러리입니다.

```
pip install python-dotenv
```

사용 예시입니다.

```
from dotenv import load_dotenv

load_dotenv()
```

11.3 os.getenv

Python에서는 `os.getenv`로 환경변수를 읽을 수 있습니다.

```
import os
from dotenv import load_dotenv

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")
database_url = os.getenv("DATABASE_URL")
```

환경변수가 없을 때 기본값을 줄 수도 있습니다.

```
debug = os.getenv("DEBUG", "false")
```

11.4 OPENAI_API_KEY

AI API를 호출할 때는 API Key를 코드에 직접 쓰지 말고 환경변수로 읽습니다.

```
import os

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

if OPENAI_API_KEY is None:
    raise RuntimeError("OPENAI_API_KEY가 설정되지 않았습니다")
```

이렇게 하면 실수로 key가 GitHub에 올라가는 위험을 줄일 수 있습니다.

11.5 DATABASE_URL

DB 연결 문자열도 환경변수로 관리하는 경우가 많습니다.

```
DATABASE_URL = os.getenv("DATABASE_URL")
```

개발 환경과 운영 환경은 DB 주소가 다를 수 있습니다. 코드 안에 고정하지 않고 환경변수로 분리하면 환경별 설정이 쉬워집니다.

12. 예외 처리

API 서버는 항상 정상 입력만 받지 않습니다. 잘못된 요청, 없는 데이터, 외부 API 실패, 파일 처리 오류가 생길 수 있습니다. 이런 상황을 response로 잘 표현하는 것이 예외 처리입니다.

12.1 try / except

Python에서 오류를 잡을 때는 `try / except`를 씁니다.

```
try:
    result = risky_function()
except Exception as error:
    print(error)
```

FastAPI endpoint 안에서도 사용할 수 있습니다.

```
@app.get("/divide")
def divide(a: int, b: int):
    try:
        return {"result": a / b}
    except ZeroDivisionError:
        return {"error": "0으로 나눌 수 없습니다"}
```

하지만 API에서는 단순히 {"error": ...}를 반환하는 것보다 적절한 status code를 함께 반환하는 것이 좋습니다.

12.2 HTTPException

FastAPI에서는 `HTTPException`으로 에러 응답을 만들 수 있습니다.

```
from fastapi import HTTPException

@app.get("/posts/{post_id}")
def read_post(post_id: int):
    post = find_post(post_id)

    if post is None:
        raise HTTPException(status_code=404, detail="게시글을 찾을 수 없습니다")

    return post
```

`raise`를 만나면 함수 실행이 중단되고, FastAPI가 에러 응답을 만들어 보냅니다.

12.3 status_code

`status_code`는 어떤 HTTP 상태 코드로 응답할지 정합니다.

```
raise HTTPException(status_code=400, detail="잘못된 요청입니다")
```

상태 코드를 숫자로 직접 써도 되지만, `fastapi.status`를 쓰면 의미가 더 명확합니다.

```
from fastapi import HTTPException, status

raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="게시글을 찾을 수 없습니다",
)
```

12.4 detail

`detail`은 클라이언트에게 전달할 에러 설명입니다.

```
raise HTTPException(status_code=400, detail="message는 필수입니다")
```

응답은 보통 아래처럼 보입니다.

```
{
  "detail": "message는 필수입니다"
}
```

React에서는 이 `detail`을 읽어 사용자에게 보여줄 수 있습니다.

12.5 에러 처리 예시: 채팅 API

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    message = request.message.strip()

    if not message:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="message는 비어 있을 수 없습니다",
        )

    try:
        answer = f"입력한 메시지: {message}"
        return {"answer": answer}
    except Exception:
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="채팅 응답 생성 중 오류가 발생했습니다",
        )
```

입문 단계에서는 모든 오류를 `except Exception`으로 덮어버리는 습관은 조심해야 합니다. 오류 원인을 로그로 남기고, 클라이언트에는 필요한 정도의 메시지만 전달하는 것이 좋습니다.

13. React와 연결하는 FastAPI 예제

이제 사용자가 공부한 React와 FastAPI를 연결해 봅시다.

13.1 FastAPI 코드

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```

)

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    answer: str

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest):
    return {"answer": f"FastAPI가 받은 메시지: {request.message}"}

```

실행합니다.

```
uvicorn main:app --reload
```

13.2 React fetch 코드

```

async function sendMessage(message) {
  const response = await fetch("http://localhost:8000/chat", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ message }),
  });

  if (!response.ok) {
    throw new Error("API 요청 실패");
  }

  const data = await response.json();
  return data.answer;
}

```

React가 보내는 JSON은 아래와 같습니다.

```

{
  "message": "안녕"
}

```

FastAPI가 돌려주는 JSON은 아래와 같습니다.

```

{
  "answer": "FastAPI가 받은 메시지: 안녕"
}

```

13.3 연결이 안 될 때 확인할 것

React와 FastAPI 연결에서 자주 확인할 것은 아래입니다.

- FastAPI 서버가 localhost:8000에서 실행 중인지 확인합니다.
- React 요청 URL이 http://localhost:8000/chat처럼 정확한지 확인합니다.
- HTTP method가 FastAPI decorator와 맞는지 확인합니다.

- JSON body의 key가 Pydantic 모델 필드와 맞는지 확인합니다.
- Content-Type: application/json header가 있는지 확인합니다.
- CORS 설정에 http://localhost:5173이 들어 있는지 확인합니다.
- 422가 나오면 request body 모양을 먼저 확인합니다.

14. 작은 완성 예제: 게시글 API

이 예제는 GET, POST, path parameter, query parameter, request body, HTTPException을 한 번에 연습합니다.

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel

app = FastAPI()

class PostCreate(BaseModel):
    title: str
    content: str

posts = [
    {"id": 1, "title": "첫 글", "content": "FastAPI 시작"},
    {"id": 2, "title": "두 번째 글", "content": "React 연결"},
]

@app.get("/posts")
def list_posts(keyword: str = ""):
    if keyword == "":
        return posts

    return [
        post for post in posts
        if keyword.lower() in post["title"].lower()
    ]

@app.get("/posts/{post_id}")
def read_post(post_id: int):
    for post in posts:
        if post["id"] == post_id:
            return post

    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail="게시글을 찾을 수 없습니다",
    )

@app.post("/posts", status_code=status.HTTP_201_CREATED)
def create_post(post: PostCreate):
    new_post = {
        "id": len(posts) + 1,
        "title": post.title,
        "content": post.content,
    }
    posts.append(new_post)
    return new_post

@app.delete("/posts/{post_id}")
def delete_post(post_id: int):
    for index, post in enumerate(posts):
```

```

if post["id"] == post_id:
    deleted = posts.pop(index)
    return {"deleted": deleted}

raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="게시글을 찾을 수 없습니다",
)

```

테스트할 요청은 아래와 같습니다.

```

GET /posts
GET /posts?keyword=React
GET /posts/1
POST /posts
DELETE /posts/1

```

이 예제는 실제 DB를 쓰지 않고 메모리의 `posts` 리스트를 사용합니다. 서버를 재시작하면 데이터가 초기화됩니다. DB를 붙이면 이 리스트 부분이 SQLAlchemy나 ORM 코드로 바뀝니다.

15. RAG 입문 관점에서 보는 FastAPI 흐름

사용자가 RAG를 만들고 싶다면 FastAPI는 "파일을 받고, 문서를 읽고, 질문을 받아 답변을 돌려주는 API 서버" 역할을 합니다.

15.1 필요한 endpoint

RAG 입문 프로젝트에서는 보통 아래 endpoint가 필요합니다.

```

POST /upload-pdf  PDF 업로드
POST /chat        질문 전송
GET /documents    업로드한 문서 목록 조회
DELETE /documents/{document_id} 문서 삭제

```

15.2 PDF 업로드 흐름

```

React에서 FormData로 PDF 전송
FastAPI에서 UploadFile로 받음
서버에 파일 저장
PDF 텍스트 추출
문단으로 나누기
embedding 생성
vector DB 저장

```

15.3 질문 응답 흐름

```

React에서 질문을 JSON으로 전송
FastAPI에서 ChatRequest로 검증
질문 embedding 생성
vector DB에서 관련 문단 검색
LLM에 관련 문단과 질문 전달
JSON으로 답변 반환

```

FastAPI 입문 단계에서는 모든 RAG 구현을 한 번에 끝내려고 하기보다 아래 순서로 연습하는 것이 좋습니다.

1. /health 만들기
2. /chat에서 받은 메시지 그대로 반환하기
3. React에서 /chat 호출하기
4. /upload-pdf로 파일 업로드하기
5. PDF 텍스트 길이만 반환하기
6. 추출된 텍스트를 저장하고 검색하기
7. LLM API와 연결하기

16. 자주 만나는 오류

16.1 404가 나옴

요청한 URL과 FastAPI decorator의 path가 같은지 확인합니다.

```
@app.post("/chat")
```

이 endpoint는 아래 요청으로 호출해야 합니다.

```
POST /chat
```

GET /chat으로 호출하면 method가 달라서 원하는 endpoint가 실행되지 않습니다.

16.2 422가 나옴

Pydantic 모델과 요청 JSON이 맞는지 확인합니다.

```
class ChatRequest(BaseModel):  
    message: str
```

정상 요청입니다.

```
{"message": "안녕"}
```

문제가 되는 요청입니다.

```
{"text": "안녕"}
```

필드 이름이 `message`가 아니라 `text`라서 검증에 실패합니다.

16.3 CORS 오류가 나옴

React 개발 서버 주소를 확인합니다.

```
http://localhost:5173
```

그리고 FastAPI CORS 설정에 같은 주소가 있는지 확인합니다.

```
allow_origins=["http://localhost:5173"]
```

16.4 500이 나옴

서버 내부 오류입니다. 터미널 로그를 확인해야 합니다.

```
Traceback ...
```

500은 브라우저 화면보다 FastAPI 서버를 실행한 터미널 로그가 더 중요합니다.

16.5 Swagger UI에서는 되는데 React에서는 안 됨

이 경우 CORS, 요청 URL, header, body 직렬화를 의심합니다.

React에서 JSON을 보낼 때는 아래 세 가지가 맞아야 합니다.

```
method: "POST"
headers: {
  "Content-Type": "application/json",
}
body: JSON.stringify({ message: "안녕" })
```

16.6 파일 업로드가 안 됨

파일 업로드는 JSON이 아니라 `FormData`입니다.

```
const formData = new FormData();
formData.append("file", file);
```

그리고 FastAPI 매개변수 이름과 `append`의 key가 같아야 합니다.

```
async def upload_file(file: UploadFile = File(...)):
    ...
```

```
formData.append("file", file);
```

17. 학습 순서

이 문서의 키워드를 실제 공부 순서로 바꾸면 아래와 같습니다.

1. Python 함수, 매개변수, return
2. 타입 힌트: str, int, bool, list, dict
3. Optional, Union
4. class, instance, `__init__`, attribute, method, 상속
5. async, await, 동기 함수, 비동기 함수
6. client, server, request, response
7. URL, endpoint, HTTP method
8. GET, POST, PUT, PATCH, DELETE
9. status code와 200, 201, 400, 401, 403, 404, 422, 500
10. header, body, JSON
11. query parameter, path parameter
12. FastAPI app, `@app.get`, `@app.post`
13. uvicorn, reload, Swagger UI
14. request body, response body, JSON response
15. Pydantic BaseModel, validation
16. path/query/body/header/cookie parameter

17. CORS, CORSMiddleware, allow_origins
18. UploadFile, File, multipart/form-data
19. 파일 저장, PDF 읽기
20. .env, python-dotenv, os.getenv
21. OPENAI_API_KEY, DATABASE_URL
22. HTTPException, status_code, detail, try/except

처음부터 완벽하게 이해하려고 하지 않아도 됩니다. FastAPI는 작은 endpoint를 만들고 Swagger UI에서 직접 눌러 보면서 배울 때 가장 빨리 익숙해집니다.

18. 최종 체크리스트

아래 질문에 답할 수 있으면 FastAPI 기본 개념은 한 번 정리된 것입니다.

- `def chat(message: str) -> dict`를 읽을 수 있는가?
- path parameter와 query parameter의 차이를 설명할 수 있는가?
- request body를 Pydantic BaseModel로 받는 이유를 설명할 수 있는가?
- 422 Validation Error가 왜 생기는지 대략 판단할 수 있는가?
- React 개발 서버 localhost:5173과 FastAPI 서버 localhost:8000의 CORS 관계를 이해하는가?
- UploadFile과 FormData가 한 쌍으로 쓰인다는 것을 아는가?
- API Key를 코드에 직접 쓰지 않고 .env와 os.getenv로 읽을 수 있는가?
- HTTPException(status_code=404, detail="...")의 의미를 설명할 수 있는가?

참고 자료

- FastAPI 공식 문서: <https://fastapi.tiangolo.com/>
- FastAPI First Steps: <https://fastapi.tiangolo.com/tutorial/first-steps/>
- FastAPI Request Body: <https://fastapi.tiangolo.com/tutorial/body/>
- FastAPI Path Parameters: <https://fastapi.tiangolo.com/tutorial/path-params/>
- FastAPI Query Parameters: <https://fastapi.tiangolo.com/tutorial/query-params/>
- FastAPI CORS: <https://fastapi.tiangolo.com/tutorial/cors/>
- FastAPI Request Files: <https://fastapi.tiangolo.com/tutorial/request-files/>
- FastAPI Handling Errors: <https://fastapi.tiangolo.com/tutorial/handling-errors/>
- FastAPI Environment Variables: <https://fastapi.tiangolo.com/environment-variables/>
- Pydantic 공식 문서: <https://docs.pydantic.dev/>
- Python typing 공식 문서: <https://docs.python.org/3/library/typing.html>